

Cell-Guard — Product Requirements Document

2026-09-17 · @Someone

Overview & Problem Statement

Cell-Guard is a policy-enforcement gateway that sits between an AI agent and the tools it's allowed to call. Every tool call the agent attempts is checked against a set of Cedar authorization policies before it executes — not by asking the model to behave, but by putting a deterministic check in front of it that the model cannot talk its way around.

The problem. AI agents are increasingly given write access: they can approve expenses, edit records, send money, delete data. The instructions that drive that behavior come from text the agent reads — a chat message, a document, a webpage. That text is not trusted input. A prompt injection attack hides an instruction inside content the agent processes, and if there is no check between "the model decided to do X" and "X happens," the injected instruction executes with the same authority as a legitimate one.

Most teams building agents right now rely on the system prompt ("don't do anything harmful") as the only safeguard. That is not a security boundary — it is a suggestion the model can be talked out of. Cell-Guard demonstrates the alternative: authorization enforced outside the model's reasoning, where a forbidden action stays forbidden no matter what the agent was convinced to attempt.

Why now. Agentic AI with tool access is moving into production faster than the authorization patterns for it have matured. This is a small, concrete demonstration of a gap that will only get more expensive to ignore.

Goals & Success Metrics

Primary goal: ship a working, deployed demonstration that an injected instruction can be blocked deterministically, and make that block visible on screen within 3 minutes.

Success criteria for this submission:

1. The same attack payload produces two different outcomes: it succeeds against the agent alone (LOG_ONLY), and it fails against the agent + gateway (ENFORCE) — both shown live, not described.

2. All 3 tool calls (legitimate and malicious) are visible in an audit trail with a clear ALLOW/DENY decision and reason.
3. The system is deployed on AWS with a working URL, not just running locally.
4. Both team members can independently explain and defend their half of the system to a judge.
5. Submission includes the demo video, working repo, and Builder Center blog post — all three required, none rushed.

Non-goal: breadth. One clean attack, cleanly stopped, beats three half-finished scenarios.

Target Users & Use Case

Who this is for, in reality: any team shipping an AI agent with write access to money, data, or infrastructure — a support bot that can issue refunds, an ops agent that can restart services, an internal assistant that can edit records. Today most of these ship with the system prompt as the only line of defense.

Demo persona: a finance-ops assistant used internally at a company. Employees chat with it to get routine tasks done — checking expense status, looking up a customer record, approving small reimbursements.

Demo scenario: a message is submitted to the agent that looks like a normal customer note but contains a hidden instruction: approve a reimbursement far above the normal limit, or delete a customer record. Without Cell-Guard, the agent complies because nothing stops it. With Cell-Guard, the Cedar policy evaluates the actual parameters of the attempted action (amount, action type) independent of what the agent was told, and blocks it.

Scope

Scope is deliberately narrow. A 4-day build with a 2-person team fails by being ambitious, not by being small.

In scope:

- One agent (Bedrock, tool-calling)
- One gateway (API Gateway + Lambda) enforcing Cedar policy via Amazon Verified Permissions

- Three tools: `approve_expense`, `delete_customer_record`, `send_wire_transfer`
- One attack scenario, demonstrated in both LOG_ONLY and ENFORCE modes
- One audit log (DynamoDB)
- One frontend chat UI

Out of scope (explicitly, so nobody drifts into building it):

- Multi-agent delegation chains or swarm topologies
- Cryptographic capability attestation between agents
- TLA+ formal verification of the orchestration harness
- Multi-tenant / hub-and-spoke account architecture
- Shuffle sharding or cell-based infrastructure isolation
- Any policy authoring UI — policies are written directly in AVP

Everything in the out-of-scope list is a legitimate enterprise concern and a bad idea for this weekend. If either track finishes early, polish what's in scope before reaching for anything on this list.

Functional Requirements

Agent

1. Accepts a chat message from the user and from a simulated "customer note" input (the injection vector).
2. Calls Bedrock with tool-calling enabled against the 3 defined tools.
3. Sends every tool call attempt to the gateway endpoint before execution — the agent never calls a tool Lambda directly.
4. Displays the gateway's decision (ALLOW/DENY) and reason back to the user in the chat UI.

Tools (each a Lambda + DynamoDB write) 5. `approve_expense(amount, employee_id)` — writes an approval record if allowed. 6. `delete_customer_record(customer_id)` — deletes a record if allowed. 7. `send_wire_transfer(amount, recipient)` — writes a transfer record if allowed. 8. Each tool only executes after receiving an ALLOW from the gateway — never before.

Gateway & policy 9. Every tool call request is evaluated against Cedar policies in Amazon Verified Permissions before the tool executes. 10. At minimum: forbid `approve_expense` above a fixed threshold; forbid `delete_customer_record` unconditionally for this demo;

forbid `send_wire_transfer` above a fixed threshold. 11. Supports two modes: LOG_ONLY (evaluates and logs, never blocks) and ENFORCE (evaluates and blocks). Mode must be switchable without a code deploy. 12. Every decision (tool, params, principal, ALLOW/DENY, reason, timestamp) is written to the audit log table.

Frontend 13. Chat interface showing the conversation, the tool the agent attempted, and the resulting decision. 14. A visible mode indicator (LOG_ONLY vs ENFORCE) so the demo video can show the flip clearly.

System Architecture

Components

- Frontend: static chat UI on Amplify Hosting (or S3 + CloudFront)
- Agent Lambda: orchestrates Bedrock tool-calling
- API Gateway: single `/invoke-tool` route
- Gateway Lambda: receives the tool-call request, calls Amazon Verified Permissions `IsAuthorized`, then either invokes the target tool Lambda or returns a DENY
- 3 tool Lambdas: `approve_expense`, `delete_customer_record`, `send_wire_transfer`
- DynamoDB — two tables: application data (expenses/customers/transfers) and a separate audit log
- Amazon Verified Permissions: hosts the Cedar schema and policies

The contract between agent and gateway (this is the seam the two-person split is built around):

```
POST /invoke-tool
Request: { "tool": "approve_expense" | "delete_customer_record" |
"send_wire_transfer",
          "params": {...tool-specific},
          "principal": { "user_id": "string", "role": "agent" },
          "context": { "session_id": "string", "source": "chat" |
"inject_doc" } }
Response: { "decision": "ALLOW" | "DENY", "reason": "string", "result": {...}
} // result only if ALLOW
```

The agent never calls a tool Lambda directly — every path to a tool runs through this one endpoint, which is what makes the enforcement real rather than cosmetic.

Non-Functional Requirements

- **Cost:** use a small/cheap Bedrock model — credits (\$100/team plus new-account credits) must cover the full weekend, including retakes during demo recording.
- **Latency:** the gateway check adds a round trip before every tool call; keep it fast enough that the demo doesn't stall (a few hundred ms is fine, seconds is not).
- **Security posture:** least-privilege IAM on every Lambda — no Lambda gets permissions it doesn't use. The gateway Lambda's own AWS permissions should not be the thing granting the risky action; Cedar's decision is the gate, not IAM alone.
- **Reliability of the demo path:** the exact sequence shown in the video (happy path → attack in LOG_ONLY → attack in ENFORCE) must be rehearsed and reproducible on demand, not something that "usually works."

Judging Criteria Alignment

Criterion	How Cell-Guard scores on it
Idea & Impact	Framed as a personal, concrete failure mode — any AI agent with write access is vulnerable to this today — not an abstract infrastructure thesis.
Built on AWS	Mandatory to win, and central to the build: Bedrock, Lambda, API Gateway, DynamoDB, Amazon Verified Permissions. Ship It track — deployed, with a URL.
Learning	Both teammates will have shipped something neither had built before: agent tool-calling with Bedrock, and Cedar-based authorization with AVP.
Execution	Deliberately narrow scope (see Scope section) so the demo path is fully working, not partially built.
Demo video	The LOG_ONLY → ENFORCE contrast is a strong, visual, 3-minute story — built around this from day one rather than improvised at the end.

Team Ownership & Work Split

Split along the one real seam in this architecture: the agent doesn't need to know how the gateway enforces policy, and the gateway doesn't need to know how the agent thinks. Both sides build against the locked API contract above and nothing else.

Person A — Agent & Frontend. Owns the Bedrock agent, the tool-calling logic, the chat UI, and the injected attack payload. Builds against a mock `/invoke-tool` that always returns ALLOW until the real gateway is ready, then swaps the URL.

Person B — Gateway & Policy. Owns the Cedar schema and policies in Amazon Verified Permissions, the API Gateway + gateway Lambda, the 3 tool Lambdas, and the audit log. Builds and tests entirely with curl/Postman against the contract — doesn't need the agent running to verify the gateway works.

Rules that keep the two tracks from interfering:

- Separate repo folders (`/agent-frontend` , `/gateway-policy`), separate branches, PRs to merge.
- Person-prefixed AWS resource names (`cellguard-agent-*` vs `cellguard-gateway-*`) — no shared Lambda, no shared IAM role.
- Separate DynamoDB tables owned by whichever side needs them.
- If either person is reading the other's code to debug something, that's a sign the contract had a gap — fix the contract, don't reach into the other's files.
- The only integration point is swapping the mock URL for the real one (see Timeline).

Timeline & Milestones

Day	Person A (Agent & Frontend)	Person B (Gateway & Policy)
Thu — Kickoff	Stand up a mock <code>/invoke-tool</code> returning ALLOW; build agent tool-calling and chat UI against it	Write the Cedar schema + policies in AVP; stand up the real API Gateway + Lambda matching the contract
Fri — Core build	Finish tool-calling against all 3 tools (still on mock); write and test the attack payload in isolation; build the full chat UI	Build the 3 tool Lambdas + their DynamoDB table; wire the gateway Lambda to AVP's <code>IsAuthorized</code> ; log every decision to the audit table; test with curl/Postman

Day	Person A (Agent & Frontend)	Person B (Gateway & Policy)
Sat — Integration	Swap the mock URL for the real gateway URL — the one line that should change; debug together if it doesn't just work; polish UI after	Run LOG_ONLY → confirm attack logged, not blocked; flip to ENFORCE → confirm it's blocked; script the mode toggle for the video
Sun — Submit	Record the 3-minute demo; draft the agent-side half of the blog post	Final clean deploy check; draft the policy-side half of the blog post; on call during recording

Milestone: by end of Friday, both tracks should be independently demoable — Person A's agent working against the mock, Person B's gateway working against curl. Saturday morning's swap is a connection, not a first integration.

Risks & Mitigations

Risk	Mitigation
Bedrock model access not yet approved on the AWS account	Request access the moment registration is done, Thursday morning at the latest — don't wait for kickoff
Cedar/AVP setup takes longer than expected	Start it first thing Thursday, before any Lambda code; keep the policy set to 4-5 rules, not more
Contract mismatch between the two tracks (field names, casing)	Contract is written down and locked before splitting; any change to it is communicated immediately, not discovered at integration
Running out of time before the attack demo reliably works	Scope is intentionally narrow (see Scope section); the LOG_ONLY → ENFORCE toggle is treated as the critical path, built and rehearsed before any polish work
Live demo doesn't reproduce on the first take while recording	Rehearse the full sequence at least twice before recording; Person B on call during recording in case the gateway needs a restart

Risk	Mitigation
Running low on AWS credits from retries/redeploys	Use a small Bedrock model; tear down and redeploy cleanly only once, on Sunday, not repeatedly

Demo & Submission Plan

3-minute video structure:

1. (0:00–0:20) What it is, one sentence: an AI agent with write access, and a gateway that stops it from being talked into doing something destructive.
2. (0:20–0:50) Show the happy path — a normal request, agent calls a tool, gateway allows it, audit log records it.
3. (0:50–1:30) Show the attack in LOG_ONLY: the injected instruction gets the agent to attempt the risky action, and it succeeds — logged, not blocked. This is the "here's the gap" moment.
4. (1:30–2:15) Flip to ENFORCE, run the exact same attack, show it blocked — the decision and reason visible in the UI or log. This is the centerpiece.
5. (2:15–2:45) Quick architecture callout: what's running on AWS (Bedrock, Lambda, AVP), and what each person built.
6. (2:45–3:00) What we learned, in one line each.

Submission checklist:

- Deployed URL (Ship It requirement)
- Public repo, both folders included
- 3-minute demo video
- Builder Center blog post (counts toward the Top 5 blogs prize) — both halves merged into one post
- Submit ahead of the Sunday deadline, not at it